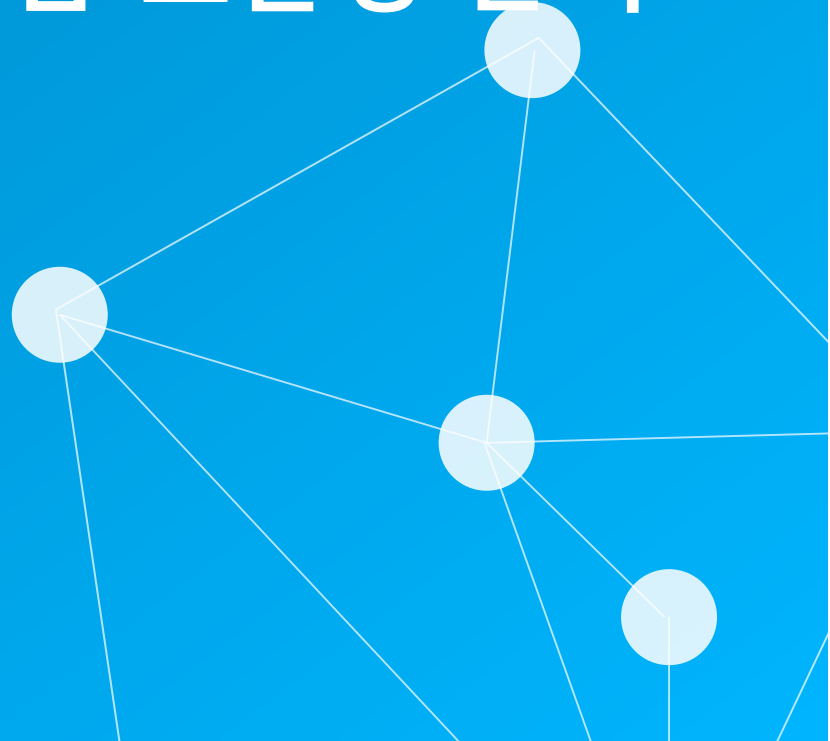


02

CHAPTER

알고리즘 효율성 분석



2장. 알고리즘 효율성 분석



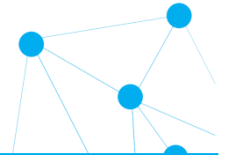
2.1 효율성 분석의 기초

2.2 점근적 성능 분석 방법

2.3 복잡도 분석 예: 반복 알고리즘

2.4 복잡도 분석 예: 순환 알고리즘

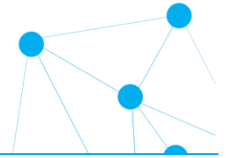
좋은 알고리즘은?



- 시간 효율성(Time efficiency)
- 공간 효율성(Space efficiency)



2.1 효율성 분석의 기초

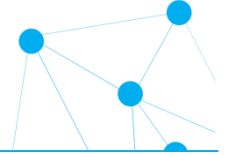


- 실제 실행시간 측정 방법 (파이썬)

```
import time                # time 모듈 불러오기
...
start = time.time()        # 현재 시각을 start에 저장(시작 시각)
testAlgorithm(input)       # 실행시간을 측정하려는 알고리즘 함수 호출
end = time.time()          # 현재 시각을 end에 저장(종료 시각)
print("실행시간 = ", end-start) # 실제 실행시간(종료-시작)을 출력
```

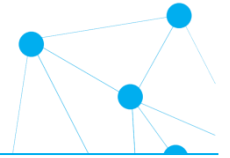
- 실행 시간 측정 방법의 문제점
 - 반드시 구현해야 함?
 - 같은 조건의 실행시간?
 - 소프트웨어 환경, 사용한 언어?
 - 모든 데이터에 대해?
- 절대적인 시간 측정 → 이론적인 복잡도 분석

알고리즘 복잡도 분석에서 중요한 점



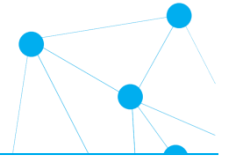
- 알고리즘에서 입력의 크기는 무엇인가?
- 복잡도에 영향을 미치는 가장 핵심적인 기본 연산은 무엇인가?
- 입력의 크기가 증가함에 따라 처리 시간은 어떤 형태로 증가하는가?
- 입력의 특성에 따라 알고리즘 효율성에는 어떤 차이가 있는가?

입력의 크기



- 알고리즘의 효율성은 **입력 크기**의 함수 형태로 표현
 - 복잡도 함수
- 무엇이 입력의 크기를 나타내는지를 먼저 명확히 결정
- 예
 - 리스트에서 어떤 값을 찾는 문제?
 - x 의 n 거듭제곱?
 - 다항식의 연산?
 - Row x Col 의 행렬 연산?
 - 그래프 연산 : 인접 행렬 표현 / 인접 리스트 표현?

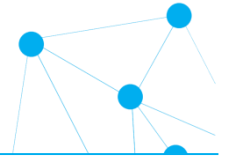
기본 연산(basic operation)



- 알고리즘에서 가장 중요한(많이 수행되는) 연산
 - 예) 다중 루프의 경우 가장 안쪽 루프에 있는 연산
 - 이 연산이 실행되는 횟수 만을 계산
- n 의 거듭제곱

	알고리즘 A	알고리즘 B	알고리즘 C
	$n * n$	$n + n + \dots + n$	$1 + 1 + \dots + 1 + 1 + 1 + \dots + 1 + \dots + 1 + 1 + \dots + 1$
	알고리즘 A	알고리즘 B	알고리즘 C
유사코드	<code>sum ← n * n</code>	1. <code>sum ← 0</code> 2. <code>for i←1 to n do</code> 3. <code>sum ← sum + n</code>	1. <code>sum ← 0</code> 2. <code>for i←1 to n do</code> 3. <code>for j←1 to n do</code> 4. <code>sum ← sum + 1</code>
전체 연산 횟수	대입 연산: 1 곱셈 연산: 1 전체 횟수: 2	대입 연산: $n + 2$ 덧셈 연산: n 전체 횟수: $2n + 1$	대입 연산: $n^2 + n + 2$ 덧셈 연산: n^2 전체 횟수: $2n^2 + n + 2$

복잡도 함수



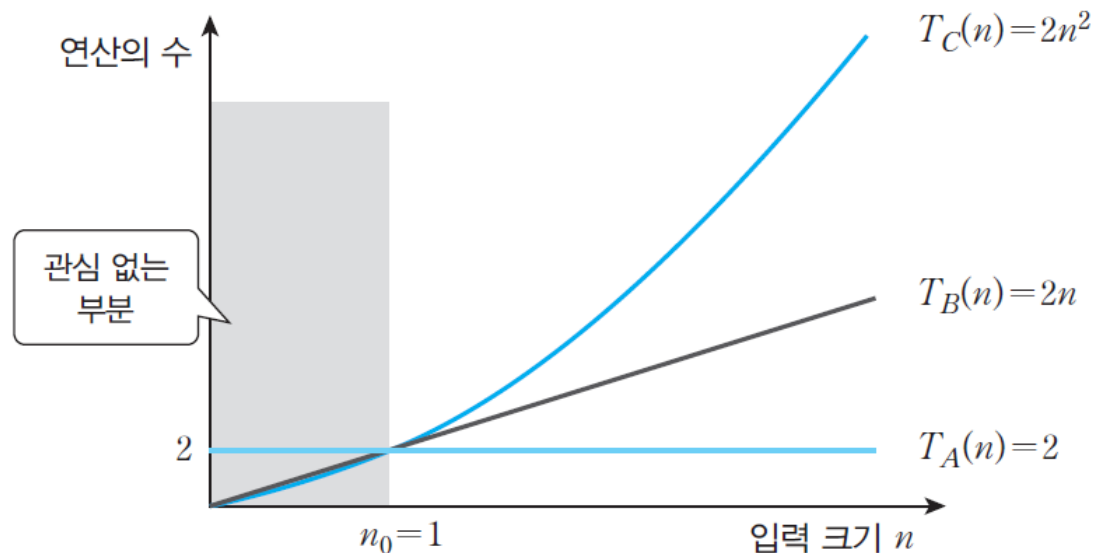
- 입력의 크기에 대한 기본 연산의 수행 횟수를 나타냄

$$T_A(n) = 2, T_B(n) = 2n, T_C(n) = 2n^2$$

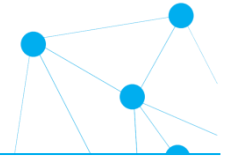
- n 이 작은 경우: 예 $n=1$

$$T_A(1) = 2, T_B(1) = 2, T_C(1) = 2$$

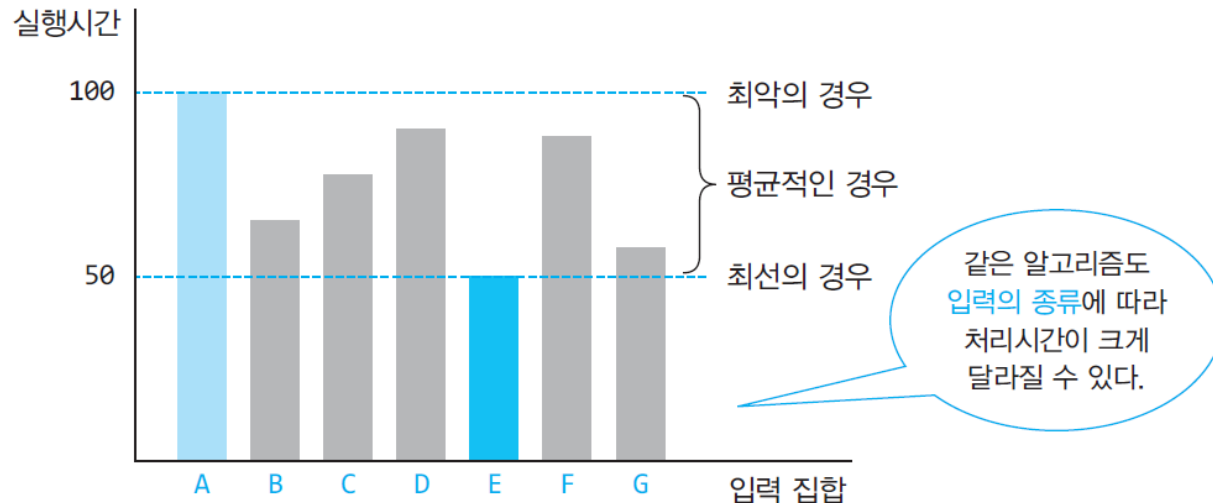
- 보통, n 이 충분히 큰 경우에만 관심 있음



최선, 최악, 평균적인 효율성

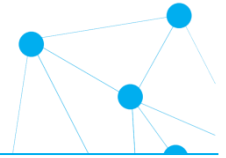


- 입력의 종류 또는 구성에 따라 다른 특성의 실행시간



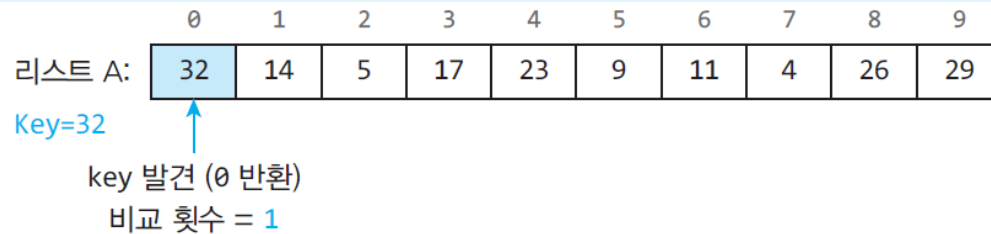
- 최선의 경우(best case): 실행시간이 가장 적은 경우를 말하는데, 알고리즘 분석에서는 큰 의미가 없다.
- 평균적인 경우(average case): 알고리즘의 모든 입력을 고려하고 각 입력이 발생할 확률을 고려한 평균적인 실행시간을 의미하는데 정확히 계산하기가 어렵다.
- 최악의 경우(worst case): 입력의 구성이 알고리즘의 실행시간을 가장 많이 요구하는 경우를 말하는데, 가장 중요하게 사용된다.

예) 순차 탐색

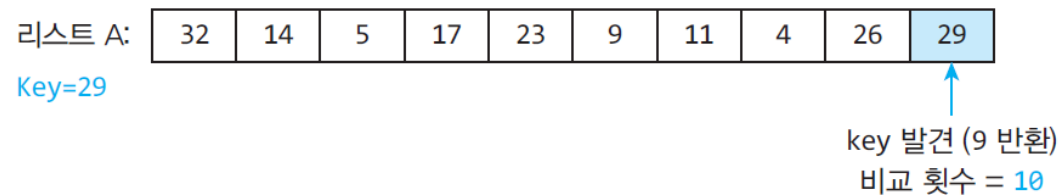


```
01 def sequential_search(A, key): # 순차 탐색. A는 리스트, key는 탐색키
02     for i in range(len(A)) :   # i : 0, 1, ... len(A)-1
03         if A[i] == key :       # 탐색 성공하면 (비교 연산. 기본 연산임)
04             return i           # 인덱스 반환
05     return -1                  # 탐색에 실패하면 -1 반환
```

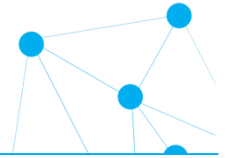
- 최선



- 최악



예) 순차 탐색



```
01 def sequential_search(A, key): # 순차 탐색. A는 리스트, key는 탐색키
02     for i in range(len(A)) :   # i : 0, 1, ... len(A)-1
03         if A[i] == key :       # 탐색 성공하면 (비교 연산. 기본 연산임)
04             return i           # 인덱스 반환
05     return -1                  # 탐색에 실패하면 -1 반환
```

- 평균: **가정**이 필요

- 리스트 내의 모든 숫자가 균일하게 탐색된다고 가정
 - Key로 사용될 가능성이 $1/n$ 으로 모든 숫자가 동일 가정
 - 비교 연산 횟수(3행)을 기준으로

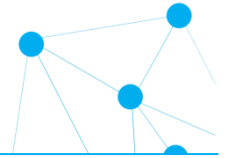
- 첫번째에서 찾은 경우 비교 연산 횟수 : 1
- 두번째에서 찾은 경우 : 2번 비교
- ...
- 마지막 n 번째에서 찾은 경우 : n 번 비교

1 ~ n 을 더하면 모든 경우에 대한 연산 횟수 총합

$$T_{avg}(n) = \frac{1+2+\cdots+n}{n} = \frac{n(n+1)/2}{n} = \frac{n+1}{2}$$

n 가지 경우에 대한 평균을 위해서 $11/31$ n 으로 나눈다.

2.2 점근적 성능 분석 방법



- 복잡도 함수에서 **차수가 가장 큰 항**이 절대적인 영향
 - 예: $T(n) = n^2 + n + 1$
 - $n=1$ 일때 : $T(n) = 1 + 1 + 1 = 3$ (n^2 항이 33.3%)
 - $n=10$ 일때 : $T(n) = 100 + 10 + 1 = 111$ (n^2 항이 90%)
 - $n=100$ 일때 : $T(n) = 10000 + 100 + 1 = 10101$ (n^2 항이 99%)
 - $n=1,000$ 일때 : $T(n) = 1000000 + 1000 + 1 = 1001001$ (n^2 항이 99.9%)

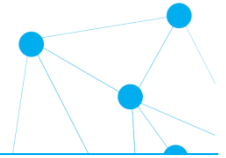
$n=100$ 인 경우

$$T(n) = n^2 + n + 1$$

99%

1%

점근적 표기(asymptotic notation)



- n 이 무한대로 커질 때의 복잡도를 간단히 표현
 - 복잡도 함수의 최고차항만을 계수 없이 취해 단순화 함

- 빅오 : 복잡도 함수의 상한 $f(n) < c * g(n)$, where $O(g(n))$

$$3n^2 + 4n \in O(n^2), \quad 2n - 3 \in O(n^2), \quad 2n(n+1) \in O(n^2)$$

$$3n^2 + 4n \notin O(n), \quad 0.000001n^3 \notin O(n^2), \quad 1000^n \in O(n!)$$

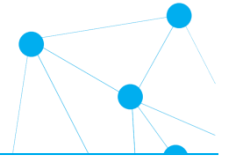
- 빅오메가 : 하한

$$2n^3 + 3n \in \Omega(n^2), \quad 2n(n+1) \in \Omega(n^2), \quad 100000n + 8 \notin \Omega(n^2)$$

- 빅세타 : 상한인 동시에 하한

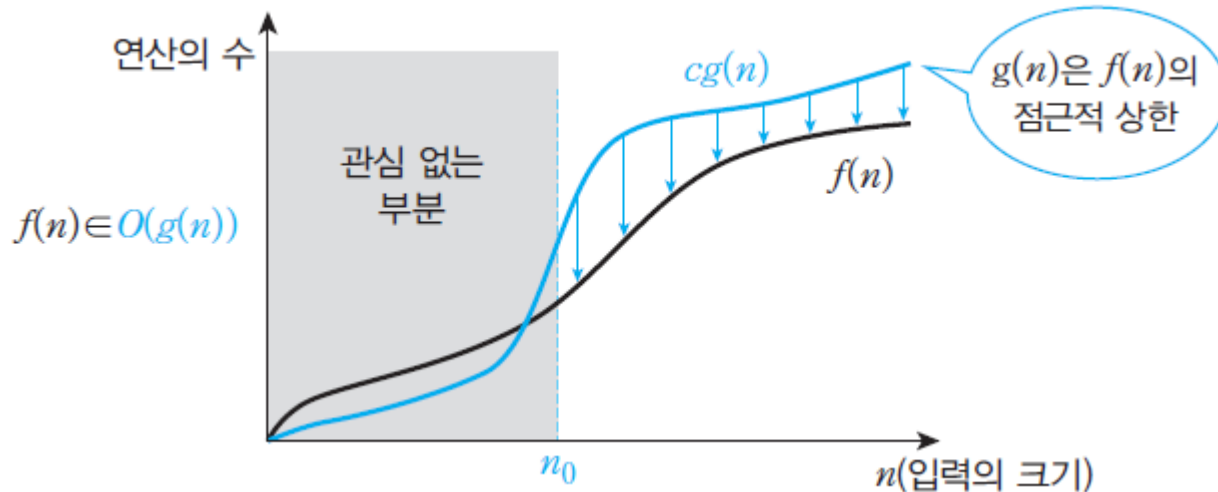
$$2n^3 + 3n \in \Theta(n^3), \quad 2n^3 + 3n \notin \Theta(n^2), \quad 2n^3 + 3n \notin \Theta(n^4)$$

빅오 표기법



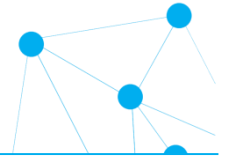
- 어떤 함수의 점근적 상한을 표시하는 방법

복잡도 함수 $f(n)$ 이 주어졌을 때 $n \geq n_0$ 인 모든 정수 n 에 대해 $|f(n)| \leq c|g(n)|$ 을 만족하는 양의 상수 c 와 자연수 n_0 가 존재하면 $f(n) \in O(g(n))$ 이다.



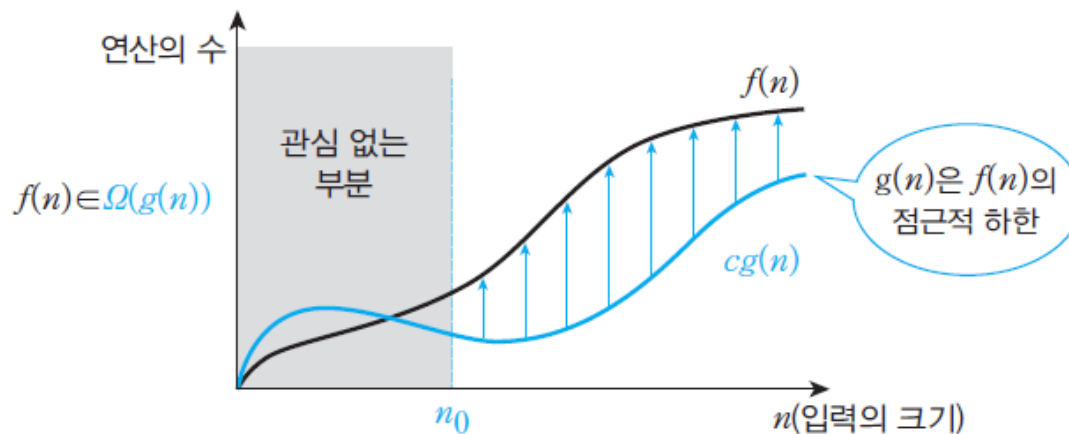
- $2n^2 + 3n + 1 \in O(n^2)$ 임을 증명해 보자. c 가 5이면 $n \geq 4$ 인 모든 n 에 대해 부등식 $|2n^2 + 3n + 1| \leq 5|n^2|$ 이 성립한다. 따라서 $c = 5$ 와 $n_0 = 4$ 가 존재하므로 이 식은 성립한다.

빅오메가 표기법



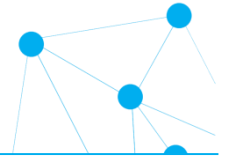
- 어떤 함수의 점근적 하한을 표시하는 방법

복잡도 함수 $f(n)$ 이 주어졌을 때 $n \geq n_0$ 인 모든 정수 n 에 대해 $|f(n)| \geq c|g(n)|$ 을 만족하는 양의 상수 c 와 자연수 n_0 가 존재하면 $f(n) \in \Omega(g(n))$ 이다.



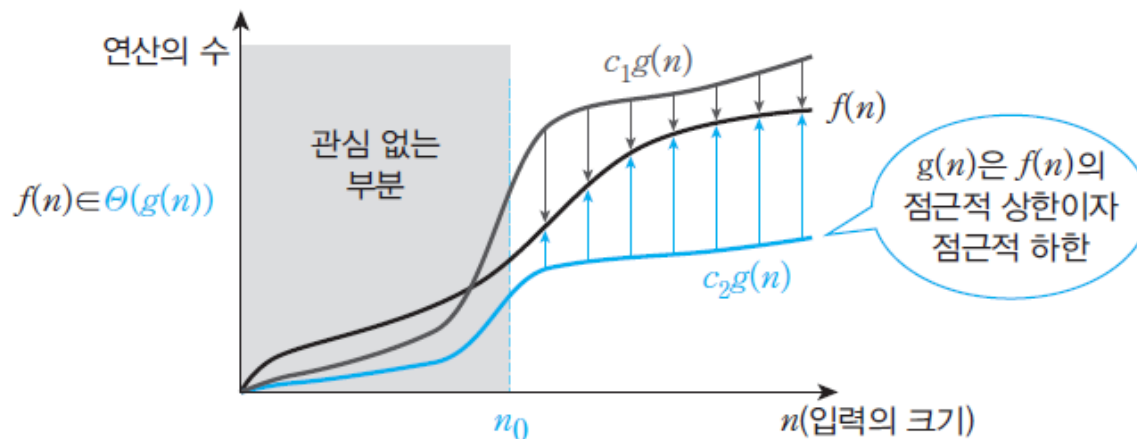
- $2n^2 + 3n + 1 \in \Omega(n^2)$ 임을 증명해 보자. c 가 2이면 $n \geq 1$ 인 모든 n 에 대해 부등식 $|2n^2 + 3n + 1| \geq 2|n^2|$ 이 성립한다. 따라서 $c = 2$ 와 $n_0 = 1$ 이 존재하므로 이 식은 성립한다.

빅세타 표기법



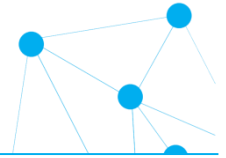
- 점근적 상한인 동시에 하한을 표시하는 방법

복잡도 함수 $f(n)$ 이 주어졌을 때 $n \geq n_0$ 인 모든 정수 n 에 대해 $c_1|g(n)| \leq |f(n)| \leq c_2|g(n)|$ 을 만족하는 양의 상수 c_1, c_2 와 자연수 n_0 가 존재하면 $f(n) \in \Theta(g(n))$ 이다.



- $2n^2 + 3n + 1 \in \Theta(n^2)$ 임을 증명해 보자. c_1 이 1이면 $n \geq 4$ 인 모든 n 에 대해 부등식 $2|n^2| \leq |2n^2 + 3n + 1|$ 이 성립한다. 또한, c_2 가 5이면 $n \geq 4$ 인 모든 n 에 대해 부등식 $|2n^2 + 3n + 1| \leq 5|n^2|$ 이 성립한다. 따라서 $c_1 = 1, c_2 = 5$ 와 $n_0 = 4$ 가 존재하므로 이 식은 성립한다.

다단계 알고리즘의 복잡도



만약 $f_1(n) \in O(g_1(n))$ 이고 $f_2(n) \in O(g_2(n))$ 이면 다음이 성립한다.

$$f_1(n) + f_2(n) \in O(\max\{g_1(n), g_2(n)\})$$

이것은 Ω -표기와 Θ -표기에서도 동일하게 적용된다.

- 예) 리스트의 중복 항목 검사

	0	1	2	3	4	5	6	7	8	9
리스트 A	32	14	5	17	23	9	11	14	26	29

중복된 항목 있음(14)

	0	1	2	3	4	5	6	7
리스트 B	22	16	14	12	7	13	27	17

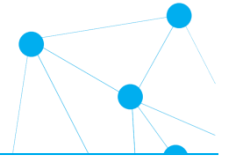
중복된 항목 없음

- 알고리즘 1: 이중 루프 사용

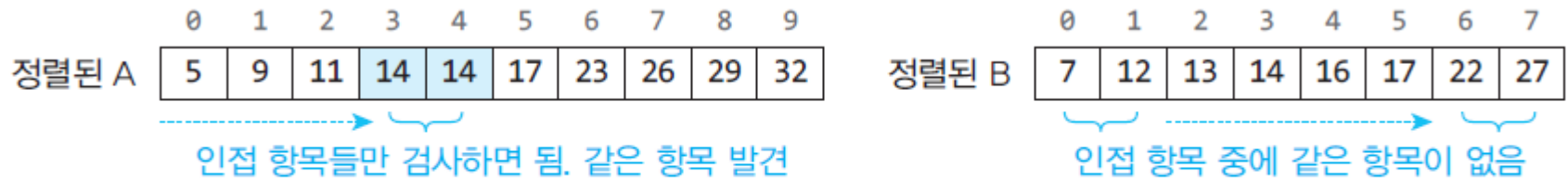
- for(i 증가: 리스트 기준)

- for(j 증가: list[i]와 비교할 대상의 index)

다단계 알고리즘의 복잡도



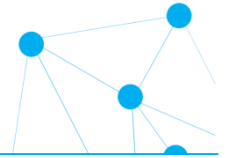
- 알고리즘 2: 2-단계 알고리즘 → 정렬 + 단일 루프



[그림 2.8] 2-단계 알고리즘: 리스트를 먼저 정렬하고, 인접 항목들만을 비교

- 복잡도 계산
 - 1단계 정렬 : $O(n \log_2 n)$ → 나중에 배운다. Quick sort는 $O(n \log_2 n)$
 - 2단계: for loop 하나만 사용 ∵ 인접 항목만 비교 → $O(n)$
 - 전체: $O(\max\{n \log_2 n, n\}) = O(n \log_2 n)$

점근적 성능 클래스들



$O(1)$: 상수형

$O(\log n)$: 로그형

$O(n)$: 선형

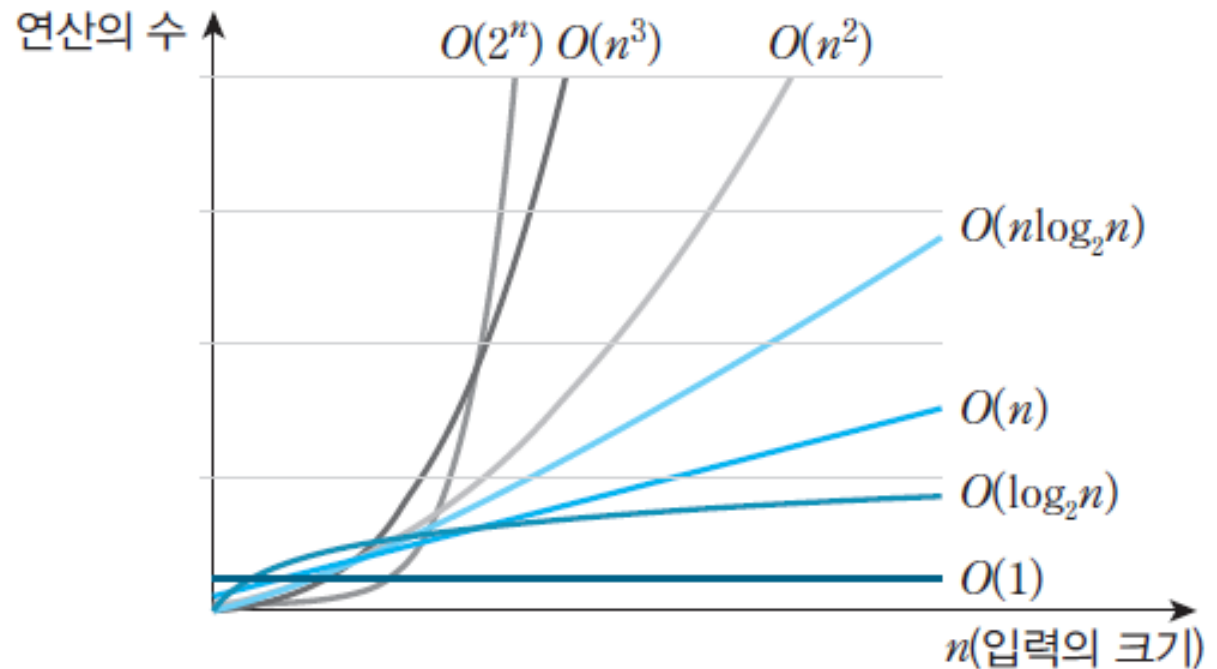
$O(n \log n)$: 선형로그형

$O(n^2)$: 2차형

$O(n^3)$: 3차형

$O(2^n)$: 지수형

$O(n!)$: 팩토리얼형

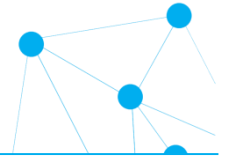


2.3 복잡도 분석 예: 반복 알고리즘



- (1) 입력의 크기를 나타내는 파라미터를 결정한다.
- (2) 기본 연산을 찾는다. 보통 반복 루프의 가장 안쪽에 있다.
- (3) 연산의 횟수가 입력 크기에 의해서만 결정되는지 살핀다. 만약 입력의 종류에 따라서도 다를 수 있다면 최선, 최악, 평균의 경우에 대해 독립적으로 복잡도를 분석해야 한다.
- (4) 기본 연산의 전체 실행 횟수를 구하는 복잡도 함수 $T(n)$ 을 구한다.
- (5) 알려진 공식 등을 이용해 $T(n)$ 을 풀고, 증가 속도를 계산한다.

자연수의 제곱 계산

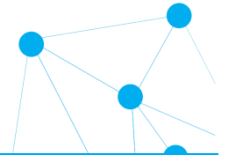


```
01 def compute_square_A(n) :  
02     return n*n
```

```
01 def compute_square_B(n) :  
02     sum = 0  
03     for i in range(n) :  
04         sum = sum + n  
05     return sum
```

```
01 def compute_square_C(n) :  
02     sum = 0  
03     for i in range(n) :  
04         for j in range(n) :  
05             sum = sum + 1  
06     return sum
```

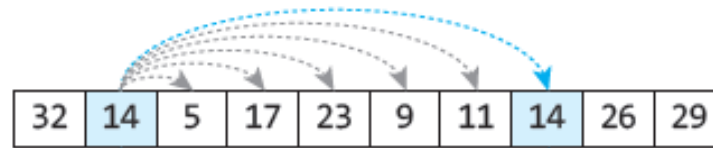
리스트의 중복 항목 탐색



- 리스트에 중복된 항목이 있는가?



중복된 항목 없음



중복된 항목 있음(14)

```
01 def unique_elements(A) :    # 리스트 A 입력
02     n = len(A)               # 입력의 크기 = 리스트의 크기
03     for i in range(n-1) :    # i : 0, 1, ... n-2
04         for j in range(i+1,n) : # j : i+1, i+2, ... n-1
05             if A[i] == A[j] : # 기본 연산
06                 return False  # 같은 항목이 있으면 False 반환
07     return True              # 같은 항목이 없으면 True 반환
```

$$\sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1$$

- 알고리즘 분석
 - 입력의 크기는? 기본 연산은?
 - 최선/최악/평균의 복잡도로 나누어 분석해야 할까?

- 최악의 경우?

$$\begin{aligned}C_{worst}(n) &= \sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1 \\&= \sum_{i=0}^{n-2} [(n-1) - (i+1) + 1] = \sum_{i=0}^{n-2} (n-1-i) \\&= \sum_{i=0}^{n-2} (n-1) - \sum_{i=0}^{n-2} i \\&= (n-1) \sum_{i=0}^{n-2} 1 - \frac{(n-2)(n-1)}{2} \\&= (n-1)^2 - \frac{(n-2)(n-1)}{2} = \frac{n(n-1)}{2} \in O(n^2)\end{aligned}$$



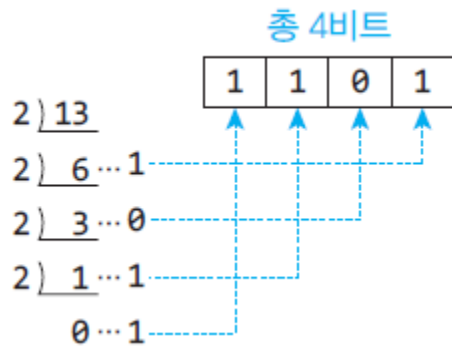
잠깐만!! 유용한 합 공식들

$$\sum_{i=1}^n 1 = n$$

$$\sum_{i=1}^n i = \frac{n(n+1)}{2}$$

$$\sum_{i=1}^n (k+i) = k \sum_{i=1}^n 1 + \sum_{i=1}^n i$$

자연수의 2진수 변환 시 비트 수 (반복 구조)



```
01 def binary_digits(n) :  
02     count = 1  
03     while n > 1 :  
04         count = count + 1  
05         n = n // 2  
06     return count
```

for, while 같은 것들

- 알고리즘 분석 (while에 n 있다고 무조건 $O(n)$ 아님 !!)
 - $n = 2^k$ 가정하면, while loop 돌 때마다 n 이 절반씩 감소

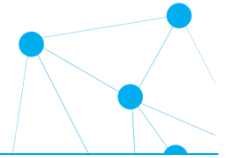
$$2^k \rightarrow 2^{k-1} \rightarrow 2^{k-2} \rightarrow \dots \rightarrow 2^1 \rightarrow 2^0$$

- 따라서 n 번 while문이 돌지 않고 k 번 돈다.

$$k = \log_2 n$$

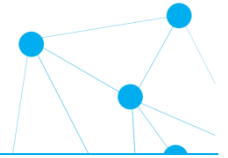
- 복잡도: $O(\log_2 n)$

2.4 복잡도 분석 예: 순환 알고리즘



- (1) 입력의 크기를 나타내는 파라미터를 결정한다.
- (2) 기본 연산을 찾는다.
- (3) 연산의 횟수가 입력 크기에 의해서만 결정되는지 살핀다.
- (4) 기본 연산의 실행 횟수를 구하기 위한 순환 관계식(recurrence relation) $T(n)$ 을 구한다.
이때, 초기 조건도 찾아야 한다.
- (5) 순환 관계식(점화식)을 풀거나 증가 속도(order of growth)를 계산한다.

팩토리얼 계산 문제 ($n!$)



- 순환 관계식, 점화식

$$F(n) = \begin{cases} 1 & n = 1 \\ F(n-1) \cdot n & n > 1 \end{cases}$$

- 순환구조

```
01 def factorial(n) :  
02     if n == 1 :  
03         return 1  
04     else :  
05         return n * factorial(n - 1)
```

n=3

```
def factorial(n) :  
    if n == 1 : return 1  
    else : return n * factorial(n - 1)
```

⑤ 6 반환

①

n=2

```
def factorial(n) :  
    if n == 1 : return 1  
    else : return n * factorial(n - 1)
```

④ 2 반환

②

n=1

```
def factorial(n) :  
    if n == 1 : return 1  
    else : return n * factorial(n - 1)
```

③ 1 반환

- 복잡도 함수의 순환 관계식

$$\begin{aligned} T(n) &= T(n-1) + 1 \\ T(1) &= 0 \end{aligned}$$

→ 곱셈 횟수 = 1

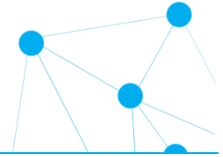
– 연속 대치법에 의한 풀이

$$\begin{aligned} T(n) &= T(n-1) + 1 \\ &= [T(n-2) + 1] + 1 = T(n-2) + 2 \\ &= [T(n-3) + 1] + 2 = T(n-3) + 3 \\ &\dots \\ &= T(n - (n-1)) + n - 1 = T(1) + n - 1 \\ &T(1) = 0 \end{aligned}$$

– 점근적 복잡도: $O(n)$

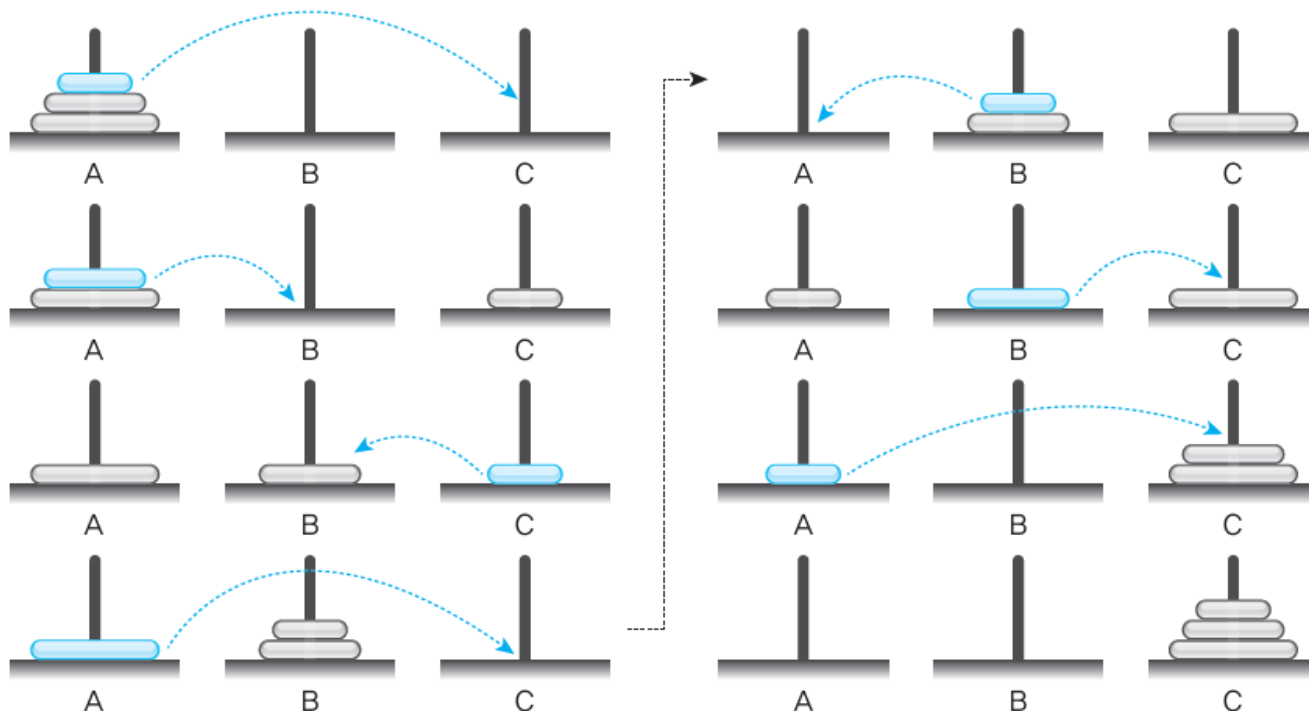
- 반복 구조의 팩토리얼 알고리즘: $O(n)$

하노이의 탑 문제



- 하노이의 탑(Tower of Hanoi) 퍼즐

- 서로 다른 n 개의 원판이 큰 것부터 아래에 놓이도록 순서대로 쌓여 있다. 이 원판들을 모두 막대 C로 옮겨라. 단, 원판은 한 번에 한 개씩만 옮길 수 있고, 작은 원판 위에 큰 원판을 놓을 수 없다. 원판의 이동 횟수는 최소로 해야 한다.



[그림 2.12] 3개의 원판을 가지는 하노이의 탑 문제의 해결 과정

- 복잡도 함수의 순환 관계식

남은 원판 하나 옮기는 연산 횟수

- $T(n) = T(n-1) + \textcircled{1} + T(n-1)$

- $T(1) = 1 \rightarrow$ 옮겨야 할 원판이 하나라면 바로 옮긴다.

- 연속 대치법에 의한 풀이 (교재 참고 p 89)

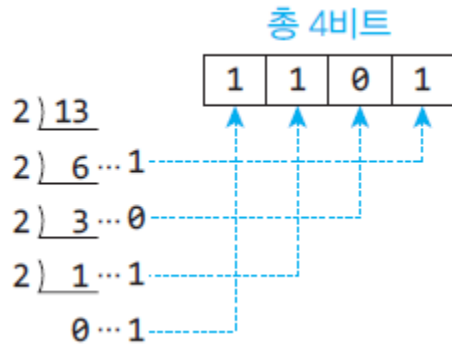
$$T(n) = 2^i T(n-i) + \underline{2^{i-1} + \dots + 2 + 1}$$

$$= 2^i T(n-i) + \underline{2^i - 1}$$

$$= 2^n - 1$$

- 복잡도: $O(2^n)$

자연수의 2진수 변환시 비트 수 (순환 구조)



```

01 def binary_digits(n) :
02     if n <= 1 :
03         return 1
04     else :
05         return 1 + binary_digits(n//2)
    
```

- 복잡도 순환 관계식

$$T(n) = T(\lfloor n/2 \rfloor) + 1 \quad \text{for } n > 1$$

$$n = 2^k$$

$$O(\log_2 n)$$

$$\begin{aligned}
 T(2^k) &= T(2^{k-1}) + 1 \\
 &= [T(2^{k-2}) + 1] + 1 = T(2^{k-2}) + 2 \\
 &= [T(2^{k-3}) + 1] + 1 = T(2^{k-3}) + 3 \\
 &\dots \\
 &= T(2^{k-k}) + k \\
 &= T(2^0) + k \\
 &= T(1) + k \\
 &= k
 \end{aligned}$$